NAVAL POSTGRADUATE SCHOOL

MONTEREY, CALIFORNIA

DISSERTATION PROPOSAL

EVALUATION MODALITY ALIGNMENT TO SYSTEMS ENGINEERING TASK TYPES: AN EMPIRICAL STUDY OF LANGUAGE MODEL DOMAIN-SPECIFIC KNOWLEDGE AND TASK-SPECIFIC EFFECTIVENESS USING DISTRACTOR VARIATION AND CONSENSUS-BASED GRADING

by

Ryan A Bell

March 2026

Dissertation Supervisor: Dr. Raymond Madachy

THIS PAGE INTENTIONALLY LEFT BLANK

Approved for public release. Distribution is unlimited.

EVALUATION MODALITY ALIGNMENT TO SYSTEMS ENGINEERING TASK TYPES: AN EMPIRICAL STUDY OF LANGUAGE MODEL DOMAIN-SPECIFIC KNOWLEDGE AND TASK-SPECIFIC EFFECTIVENESS USING DISTRACTOR VARIATION AND CONSENSUS-BASED GRADING

Ryan A Bell
Civilian, Department of the Navy
BSEE, Clemson University, 2015
MSEE, Clemson University, 2017

Submitted in partial fulfillment of the
requirements for the degree of

DOCTOR OF PHILOSOPHY IN SYSTEMS ENGINEERING
from the
NAVAL POSTGRADUATE SCHOOL
March 2026

Approved by:

Dr. Raymond Madachy Dissertation Supervisor	Dr. Raymond Madachy Dissertation Committee Chair
Dr. John Green Department of Systems Engineering	Dr. Douglas Van Bossuyt Department of Systems Engineering
Dr. Ronald Giachetti Department of Systems Engineering	Dr. Mathias Kolsch Department of Computer Science

Approved by: Oleg A. Yakimenko
Chair, Department of Systems Engineering

Approved by: Douglas Moses
Vice Provost for Academic Affairs

THIS PAGE INTENTIONALLY LEFT BLANK

ABSTRACT

The widespread adoption and availability of Large Language Models (LLMs) has opened new avenues in systems engineering, with promising capabilities in requirements analysis, design synthesis, and decision support. However, evaluating LLM performance in this domain remains methodologically under served, particularly with respect to selecting appropriate evaluation modalities for different task types. This study presents a structured framework for aligning evaluation format—specifically Multiple-Choice Questions (MCQs) and Open-Style Questions (OSQs)—to the underlying cognitive demands of systems engineering tasks. Building on the existing SysEngBench benchmark, distractor-variant MCQs are introduced to assess whether language models are relying on pattern matching or demonstrating robust understanding. In parallel, an OSQ dataset is constructed by converting each MCQ into a free-text prompt through an LLM-assisted rewriting process. OSQ responses are scored by multiple language model judges, and inter-rater agreement is used to assess evaluation reliability. Several state-of-the-art language models are benchmarked across both formats and analyze performance across INCOSE designated SE task categories. Results show that some categories (e.g., requirements analysis, risk management) are reliably evaluated by either format, while others (e.g., architectural consistency, MBSE modeling) benefit from more expressive textual evaluation or specialized grading. The resulting research informs the viability of MCQs and OSQs for the evaluation of LLMs in the domain of systems engineering, providing insights and an extensible foundation with SysEngBench that can guide the development of AI tools to determine their usefulness to practicing systems engineers.

THIS PAGE INTENTIONALLY LEFT BLANK

Table of Contents

1 Introduction **1**
1.1 Problem Statement. 1
1.2 Research Questions 2
1.3 Background 3
1.4 Research Objective 11

2 Research Approach **13**
2.1 Literature Review 13
2.2 Research Framework 14

3 Research Contributions **27**
3.1 Task-Modality Alignment Framework for Systems Engineering Evaluation . 27
3.2 Robustness Analysis of MCQ Evaluation Using Distractor Variation for Systems Engineering 27
3.3 Extension of SysEngBench with Multi-Format and Parallel Evaluation Items. 28
3.4 Comparative Effectiveness of MCQ, OSQ, and Hybrid Evaluation Modalities 28
3.5 Evidence-Based Guidance for Practical LLM Integration in Systems Engineering Workflows 28

4 Execution Plan **29**
4.1 Dissertation Outline 29
4.2 Dissertation Schedule 31
4.3 Resources and Availability Assessment 31
4.4 Notional Publications and Conference Paper 32

Acknowledgments **35**

List of References **37**

THIS PAGE INTENTIONALLY LEFT BLANK

List of Figures

Figure 1.1 Life Cycle Cost Impacts [1] 5

Figure 1.2 The Systems Engineering Vee [2] 6

Figure 2.1 Generic Evaluation IBD 15

Figure 2.2 Generic Evaluation Sequence Diagram 15

Figure 2.3 Research Approach Phases 16

Figure 2.4 Evaluation Modality Analysis Pipeline 17

Figure 2.5 SysEngBench Benchmark Mapped to INCOSE Handbook Categories [3] 18

Figure 2.6 Multiple Choice Question Style to Open-Style Question Conversion Framework [4] 19

Figure 2.7 Multiple Choice Question Style to Open-Style Question Filter Process [4] 20

Figure 2.8 Multiple Choice Question Distractor Datasets 21

THIS PAGE INTENTIONALLY LEFT BLANK

List of Tables

Table 2.1 Proposed Literature Review Outline 14

Table 2.2 Classification and Evaluation Prompts 19

Table 2.3 Notional Rubric for LLM as a Judge in Open-Style Question Evaluation 23

Table 2.4 Grading Prompt with Rubric and Structured Return Format 24

Table 4.1 Proposed Dissertation Outline 30

Table 4.2 Proposed Dissertation Schedule 31

Table 4.3 Notional Journal and Conference Publication List 32

THIS PAGE INTENTIONALLY LEFT BLANK

List of Acronyms and Abbreviations

AI Artificial Intelligence

GPT Generative Pre-trained Transformer

LLM Large Language Model

MBSE Model-Based Systems Engineering

MCQ Multiple Choice Question

OSQ Open Style Question

THIS PAGE INTENTIONALLY LEFT BLANK

CHAPTER 1:
Introduction

1.1 Problem Statement

Large Language Models (LLMs) have demonstrated the potential to revolutionize workflows in complex, technical fields such as systems engineering. LLMs exhibit strong natural language understanding and generative capabilities, enabling them to perform tasks ranging from requirements analysis to design synthesis and risk management support. However, assessing the effectiveness and performance of LLMs in such highly specialized and domain-specific applications poses significant challenges. Domain-specific benchmarks for a given field are necessary to further progress LLM performance [5], [6].

Multiple Choice Questions (MCQs) are commonly used for benchmarking due to their structured nature and ease of automatic scoring. Only one such benchmark currently exists in the systems engineering domain - SysEngBench, which provides a diverse set of MCQs that test the model's ability to understand systems engineering concepts [3], [7], [8]. However, while MCQs can provide quick insights into a model's ability to recognize predefined answers, they may not adequately evaluate the nuanced understanding and reasoning required in systems engineering [9]. Conversely, textual response formats, which require the model to generate open-ended answers and are more indicative of practical usage, may better assess deeper conceptual understanding. However, textual response formats introduce challenges related to dataset creation and evaluation methodology. There is not a textual systems engineering benchmark [8]. This gap in benchmarking shows an incomplete picture of LLM performance, limiting the systems engineering community's ability to understand LLMs' strengths and weaknesses in complex problem-solving scenarios. The gap can be filled with a study that presents a structured framework for aligning evaluation format—specifically Multiple-Choice Questions (MCQs) and Open-Style Questions (OSQs)—to the underlying cognitive demands of systems engineering tasks, as well as recording the cost associated with each benchmarking modality.

There is a critical need for a comparative evaluation framework that assesses the capabilities of LLMs using both MCQ and textual response formats in systems engineering. Such a framework should highlight not only the strengths and weaknesses of each modality but also provide insights into how these evaluation methods can complement one another in understanding LLM performance. Developing a framework to generate high-quality, domain-specific textual datasets from structured MCQs is essential. This requires maintaining the semantic fidelity of the original questions while adapting them to free text, open-ended formats, ensuring that the questions still effectively capture the complexity and nuances of systems engineering tasks.

LLMs can transform workflows and act as a force multiplier in systems engineering, enabling capabilities such as requirements analysis, design synthesis, and risk management in less time. However, assessing LLM performance in such a specialized field presents challenges, particularly due to the limited availability of domain-specific benchmarks except for SysEngBench, which focuses on MCQs. While MCQs offer structured and easily scorable evaluation methods, this research hypothesizes that MCQs fall short in capturing the deeper reasoning and conceptual understanding required in systems engineering. Textual response formats, though more representative of practical applications, introduce complexities in dataset creation, semantic fidelity, and evaluation. To address these gaps, an analysis is necessary and proposed in this research to assess LLMs using both MCQ and textual response formats, alongside methodologies for generating high-quality textual datasets.

1.2 Research Questions

The focus of this work is to explore, derive, and evaluate methodologies for benchmarking the performance of LLMs in systems engineering. Specifically, this study investigates the comparative effectiveness of MCQs and textual response formats in evaluating LLMs' understanding and reasoning capabilities in domain-specific tasks. This research also addresses challenges associated with dataset creation, evaluation metrics, and the impact of quantization on model performance. The overarching research questions are as follows:

- How do the evaluation modalities of MCQs and textual responses compare in effectively assessing the performance of LLMs in systems engineering?

- What are the strengths and limitations of using MCQs as a benchmarking methodology in a specialized domain such as systems engineering?
- How can textual response formats better capture the nuanced reasoning and conceptual understanding required for systems engineering tasks?

To address these questions, this work performed herein examines foundational challenges in benchmarking LLMs for systems engineering. The answers to the following specific questions will contribute to arriving to potential answers to the dissertation focus:

- What is the current state of benchmarking methodologies for LLMs in systems engineering?
- What is the role of textual response formats in capturing deeper conceptual understanding compared to MCQs?
- How can a systematic approach be developed to convert MCQs into high-quality textual datasets while preserving semantic fidelity?
- What evaluation metrics can effectively compare LLM-generated textual responses against reference solutions?
- How can insights gained from the comparative evaluation of MCQs and textual responses inform the design of more effective benchmarks for LLMs in systems engineering?
- How can existing systems engineering task or concept taxonomies, such as those from the INCOSE Handbook, be leveraged?

1.3 Background

Systems engineering integrates tasks like requirements analysis, design synthesis, and risk management to address complex challenges. LLMs can support these workflows by automating documentation and aiding decision-making, with a gamut of models performing well and others not so well with domain-specific tasks on tailored benchmarks such as SysEngBench that uses the MCQ evaluation modality [3]. Techniques like quantization further enhance efficiency but require careful trade-offs to maintain accuracy in high-stakes systems engineering applications [9].

1.3.1 Overview of Systems Engineering and Its Complexity

Systems engineering is a multidisciplinary approach that focuses on the design, realization, and operation of complex systems over their entire life cycles [10], [11]. Rather than working in isolation, systems engineering integrates multiple engineering domains—such as mechanical, electrical, software, and human factors engineering—into a cohesive framework. The scope of systems engineering extends beyond traditional technical concerns, incorporating elements of project management, risk assessment, stakeholder requirements, and lifecycle considerations [11], [12]. By emphasizing interdependencies among various subsystems, systems engineering ensures that the final product not only meets defined specifications but also aligns with broader organizational, regulatory, and user objectives.

The holistic nature of systems engineering sets it apart from narrowly focused engineering disciplines. While individual specialists might concentrate on a single subsystem or technology, systems engineers synthesize these efforts, ensuring that the whole configuration operates harmoniously. This integrative perspective is crucial in contexts where small changes can ripple through the entire system, affecting performance, cost, schedule, and safety [13], [14]. When designing and correcting errors, cost in particular can become a 10 to 100 times multiplier as you progress through the life cycle [15]–[18]. A systems engineer is more akin to a jack-of-all-trades instead of a subject matter expert in a singular discipline. In essence, systems engineering demands a comprehensive understanding of both technical components and human organizational structures, to successfully navigate nuanced decision-making that addresses the full breadth and depth of complex projects.

The systems engineering process dictates one begins with thorough requirements analysis, a process that elicits, documents, and validates what stakeholders need the system to accomplish. Requirements analysis ensures alignment between envisioned system capabilities and stakeholder expectations, delineating both functional and non-functional parameters. This activity is followed by design synthesis, where engineers blend various technologies, subsystems, and design principles to create a holistic architecture. Such synthesis might involve selecting appropriate materials, defining data flows, and arranging subsystems to achieve an optimal balance of performance, reliability, and maintainability. There are several frameworks that can be leveraged for architecting solutions, including but not limited to DODAF, UAF, Zachman, MagicGrid) [19]–[22]. Another aspect of the system engineer's role is program, project, or technical management that involves overseeing the execution of

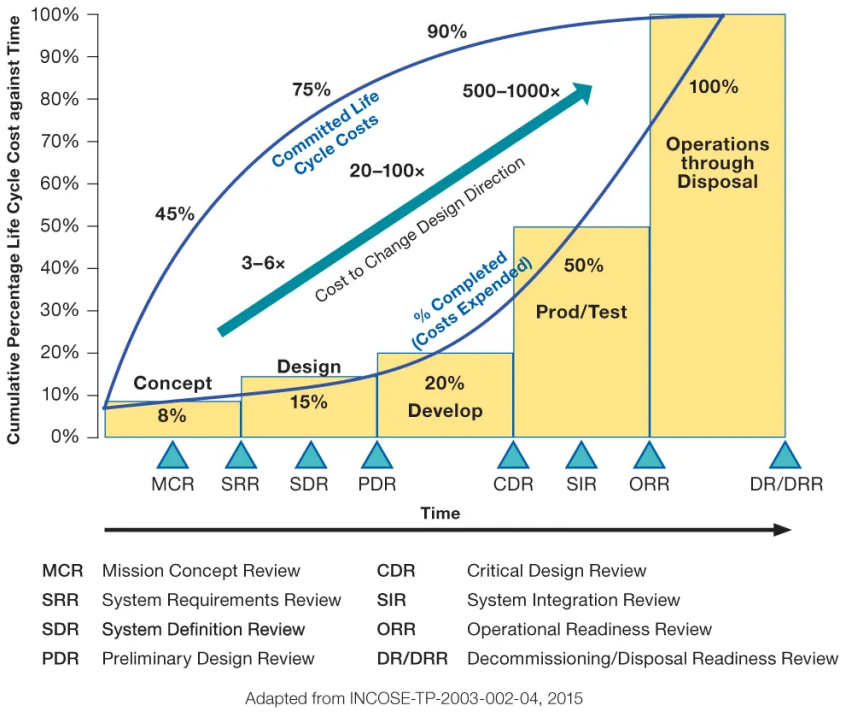

Figure 1.1. Life Cycle Cost Impacts [1]

engineering projects, ensuring that resources, timelines, and objectives align with balancing trade-offs between cost, schedule, and performance while maintaining a system's overall integrity and adaptability to evolving requirements [23]–[25].

Conceptual and abstract reasoning is common place in systems engineering as it allows practitioners to navigate complex problems before committing significant resources to detailed design and implementation, which yields the highest return [11], [13]. Rather than proceeding through trial-and-error, systems engineers develop conceptual models to anticipate interactions, identify constraints, and recognize emergent behaviors. These mental constructs enable engineers to reason abstractly about the system's architecture, performance parameters, and operational environment, ensuring that decisions made early in the project lifecycle are sound and justifiable. Building upon these abstract models, Model-Based Systems Engineering (MBSE) introduces a formalized approach that enhances the visualization and validation of system architectures before physical implementation [26]–

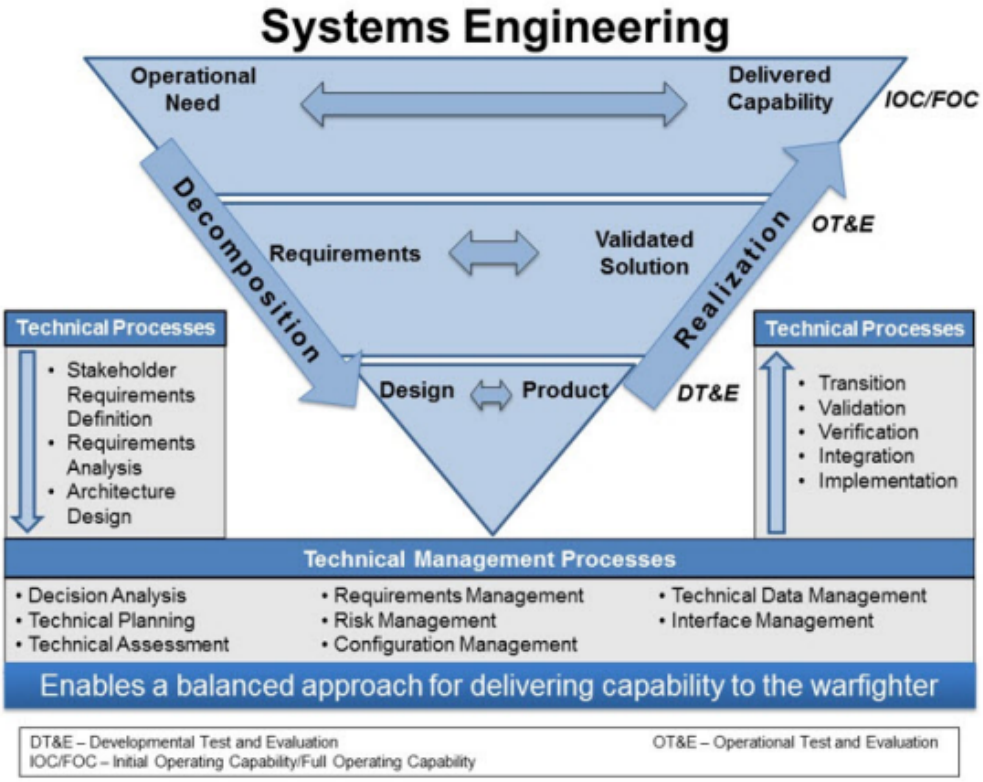

Figure 1.2. The Systems Engineering Vee [2]

[29]. MBSE provides a structured approach to developing and analyzing conceptual models through formal representations, such as SysML, enabling engineers to visualize, validate, and refine system architectures before physical implementation [30]. By leveraging MBSE, practitioners can capture the complexity of large socio-technical systems into a lower dimensional views for comprehension that enhance traceability, improve consistency, and reduce ambiguity in system design, ultimately leading to more effective decision-making throughout the system lifecycle.

As integrators of engineering disciplines, systems engineers balance competing constraints in trade-spaces, anticipate known, unknown emergent behaviors, and retain a holistic perspective to strive for a seamless convergence of technology, processes, and people to deliver valuable, resilient complex systems.

1.3.2 Introduction to Large Language Models (LLMs)

LLMs are advanced artificial intelligence systems trained on massive datasets of text to understand and generate text. These models leverage neural network architectures, particularly transformer-based architectures like Generative Pre-trained Transformer (GPT), to capture syntactic structures, semantic nuances, and pragmatic contexts [31]. When trained on an extensive corpus of text, LLMs have proven themselves capable of approaching average human performance [32]. When training on more tailored and pruned sources, small language models (SLMs) have also proven to be quite capable [33]–[35].

Such capability has effectively unlocked the potential for humans to shift their focus to more abstract, conceptual thought and leave tedious, automatable tasks to machines. The ability to quickly sort through mountains of data proves to be an indispensable tool for domains like systems engineering where legacy, document-centric processes are still heavily represented [36], [37]. The transformation from document-centric processes to model-centric processes will be heavily enabled through the usage of AI tools - leaving only a mindset shift to be required [38]–[41]

Within systems engineering, LLMs augment tasks such as requirement refinement, documentation generation, and systems modeling. For instance, LLMs can analyze initial drafts of requirements or design outlines, clarify ambiguities, improve coherence, and ensure consistent terminology. These capabilities relieve engineers of tedious editing tasks, enabling them to focus on strategic decision-making [42]

Furthermore, LLMs can support human conceptual reasoning by synthesizing large volumes of technical documents, identifying relevant design patterns, and suggesting potential architectural frameworks. While they do not replace human expertise, their ability to generate and structure large corpuses of information into digestible summaries significantly enhances productivity and coherence in complex engineering projects.

Despite their impressive capabilities, LLMs face limitations in handling complex, domain-specific challenges. A key issue is their reliance on pattern recognition rather than true understanding, which can lead to confidently asserted but incorrect outputs. In specialized contexts, LLMs may struggle to incorporate nuanced domain knowledge and produce oversimplified or irrelevant suggestions known as hallucinations [43]–[45]. Additionally,

the “black-box” nature makes trace reasoning difficult, which is problematic for critical engineering tasks although new reasoning models may remedy this pitfall [46]–[48].

1.3.3 Benchmarking as a Prerequisite for Effective Use of Language Models in Systems Engineering

Benchmarking is essential for gauging how well LLMs perform. Given the complexity and high stakes involved in designing and operating large-scale systems, it is crucial to quantitatively and qualitatively assess whether LLM outputs meet the desired standards of accuracy, reliability, and conceptual depth. By comparing LLM performance to established metrics, human experts, or other AI tools, benchmarking provides a structured mechanism to identify strengths and weaknesses in model capabilities [6], [49]–[51].

Beyond simple metrics, benchmarking also helps guide future Artificial Intelligence (AI) improvements. When engineers and researchers understand where an LLM falls short they can make informed decisions about model selection, tuning, or augmentation. Ultimately, systematic benchmarking ensures that the use of LLMs in systems engineering is grounded in evidence-based practices and aligned with industry and stakeholder needs.

Creating domain-specific benchmarks for any domain is a challenging endeavor as it requires representative datasets, carefully crafted tasks, and contextually relevant metrics. Given that systems engineering contexts are inherently diverse: what matters for a satellite design project may differ dramatically from what is critical for a nuclear submarine’s maintenance schedule. Capturing this diversity in a single benchmark demands careful thought and collaboration among experts who understand the nuances of various engineering subfields. In some cases, a single benchmark may be acceptable while in others, tailored benchmarks for for different use cases may be required.

Furthermore, the complexity of systems engineering tasks can make isolating measurable performance indicators difficult. Tasks that rely heavily on conceptual reasoning, trade-off analysis, or multi-layered stakeholder requirements prove to be challenging endeavors to reduce to simplistic performance scores. The lack of publicly available, domain-specific corpora further complicates the benchmarking process. As a result, developing robust benchmarks often involves time-consuming data collection and expert annotation to ensure that test sets remain both challenging and accurately represents systems engineering tasks.

Other fields have already begun creating domain-specific benchmarks, such as but not limited to the biomedical field [52], medical and clinical field [53], legal field [54], [55], and the financial field [56], [57].

SysEngBench represents a pioneering effort to create a tailored benchmark for evaluating LLMs in systems engineering contexts [3], [8]. By focusing specifically on the tasks, terminologies, and reasoning patterns intrinsic to the field, SysEngBench attempts to move beyond generic language benchmarks that fail to capture the complexity of systems engineering specific discourse. It seeks to provide a more dedicated measure of how well an LLM can handle domain-specific challenges.

The nature of progress in AI technologies ensures that new paradigms, methodologies, and technologies will emerge over time, potentially rendering certain aspects of the benchmark more easily performed. The extensibility requirement of the benchmark comes to light as AI technology progresses and SysEngBench must evolve to push the evaluation envelope. Nonetheless, SysEngBench lays the groundwork for a more rigorous and representative approach to evaluating LLM performance for systems engineers.

1.3.4 Evaluation Modalities in LLM Performance Testing

Multiple Choice Questions (MCQs) have long been a mainstay in educational and professional assessments due to their simplicity and objectivity. In the context of evaluating LLMs, MCQs offer a straightforward testing mechanism: a model selects from a finite set of answers, making it easy to measure correctness. This format provides a quick, quantifiable snapshot of a model's baseline knowledge and pattern-recognition abilities.

However, MCQs inherently conceal the complexity of the reasoning processes that can be assessed, meaning they offer little insight into how or why a model arrived at a particular choice. For systems engineering, the how or why is critical to evaluating competing tradespaces and picking a solution that balances competing interests.

By constraining possible answers, MCQs may oversimplify complex topics. Systems engineering scenarios that require in-depth reasoning, synthesis of multiple concepts, or consideration of subtle trade-offs are not easily captured by a handful of fixed options. This lack of nuance means MCQs may not truly reflect the model's capacity for deep under-

standing or its ability to articulate reasoning processes [58], [59]. In complex engineering environments, relying solely on MCQs risks obtaining an incomplete picture of the model's actual capabilities. Recent research suggests that providing LLMs with no correct MCQ can assist in measuring reasoning and critical thinking [60].

Textual response formats challenge LLMs to generate answers in free form, demanding a richer comprehension of the underlying material. By forgoing multiple-choice constraints, these formats encourage models to articulate reasoning, structure their responses coherently, and demonstrate the ability to connect concepts holistically. For instance, a prompt might ask the LLM to explain how a particular subsystem interacts with other components or to propose a solution that balances performance and cost.

The advantage of textual responses lies in their ability to reveal more about the model's internal "thought process" as demonstrated by reasoning models like ChatGPT o1 or DeepSeek R1 [46], [48]. Evaluating free text answers can uncover subtle reasoning errors or expose areas where the model lacks conceptual depth. While more complex to assess automatically, textual response evaluation could be aided by sophisticated metrics or even human expert review.

Despite their merits, textual responses pose significant practical challenges. Creating a dataset of open-ended questions is often more involved than formulating MCQs. It requires careful thought to ensure that prompts are unambiguous and representative of real-world tasks, while maintaining a relatively tightly bounded question. Since textual responses are not strictly right or wrong in the same way MCQ choices are, developing reliable evaluation metrics is complex. Traditional measures like accuracy scores do not directly apply. The model might produce grammatically correct but conceptually vacuous answers or resort to vague generalities that sound authoritative but lack technical rigor [59], [61]. Addressing these challenges will involve combining automated metrics with domain expert human evaluation.

1.3.5 Summary

Key takeaways:

- Systems engineering tasks demand advanced reasoning and conceptual understanding that can challenge current LLMs.
- While SysEngBench provides a foundation for benchmarking with MCQs, it lacks mechanisms for evaluating textual responses.
- A comparative study of MCQs and OSQs can address gaps in benchmarking methodologies.
- Developing robust methodologies for generating high-quality textual datasets is critical to advancing benchmarking in this domain.

1.4 Research Objective

The goal of this research is to develop an evaluation framework to inform practicing systems engineers on the viability of MCQs and OSQs for the evaluation of LLMs in systems engineering tasks. The framework will evaluate the strengths and limitations of MCQ and OSQ formats as benchmarking modalities for systems engineering tasks. The research aims to quantify how well LLMs capture the unique demands of systems engineering, including the nuanced understanding, conceptual reasoning, and problem-solving capabilities required for the field. To ensure a meaningful and fair comparison, the study will employ methodologies to transform structured MCQs into high-quality textual datasets while preserving semantic fidelity, enabling "apples-to-apples" evaluations between the two modalities.

Additionally, this research seeks to investigate the cost difference between the evaluation modalities. This involves continuing the cost modeling research done for MCQs and applying that to OSQs [9]. It would also assist in informing AI cost modeling efforts [62]–[65]. The ultimate objective is to contribute to the refinement of benchmarking methodologies for LLMs in systems engineering, providing actionable insights into their practical applications, while also addressing foundational gaps in evaluation and dataset creation. This work is expected to inform the systems engineering community on practical, cost effective usage of LLMs for systems engineering domain-specific tasks.

THIS PAGE INTENTIONALLY LEFT BLANK

CHAPTER 2:
Research Approach

This dissertation will use design and analysis systems engineering methods to explore and derive answers to the research questions.

2.1 Literature Review

The literature review within this dissertation is broken down into the relevant and comprehensive focus areas, depicted in Table 2.1. The review will provide a detailed analysis of existing benchmarks, dataset creation challenges, and evaluation modalities for LLMs.

Section ID	Chapter Title	Description
L.1	Introduction and Context Setting	• Motivation for LLMs given productivity benefits. • Summary of recent LLM advances and growing relevance to engineering. • Overview of SE complexity and representative tasks (requirements, traceability, risk). • Need for SE-specific benchmarking.
L.2	Current Benchmarking Landscape	• General benchmarks: HELM, MMLU, ARC, BIG-bench. • Reasoning benchmarks: GSM8K, ProntoQA, multi-step tasks. • Domain-specific: SWE-bench, LegalBench, MedQA, PubMedQA, Rosetta-PL. • Gaps in addressing SE-specific workflows and evaluation. • Introduction to SysEngBench structure and INCOSE Handbook categories.
L.3	Evaluation Methods for LLMs	• MCQs: strengths, classical psychometrics, distractor analysis. • OSQs: inter-rater reliability (Cohen's/Fleiss' Kappa), LLM-as-a-Judge grading. • Hybrid and purpose-built evaluation methods.
L.4	Unique Evaluation Challenges in SE and Cross-Domain Strategies	• Logical reasoning across SE views and dependencies. • Diagrammatic and formal notation reasoning (e.g., BDDs, state machines). • Numeric reasoning challenges and safety-critical applications. • Traceability, iterative refinement, and requirement consistency. • Cross-domain inspirations from law (citation grounding), software (code verification), and medicine (hallucination handling).
L.5	Experimental Designs for Benchmark Comparison	• Methods for converting MCQs to OSQs. • Statistical comparison techniques (t-test, Wilcoxon). • Case studies and lessons learned from benchmark transformations.
L.6	Measurement and Analysis Techniques	• Classical test theory: item difficulty, discrimination indices. • Item Response Theory (IRT): capability modeling. • Validity and reliability: inter-rater agreement, grounding, and test fidelity.

Table 2.1. Proposed Literature Review Outline

2.2 Research Framework

2.2.1 General Benchmarking Architecture

This section focuses on the high-level, overarching blueprint for how to benchmark different LLMs.

The evaluation of LLMs involves a structure that consists of evaluation code, a dataset to benchmark or evaluate against, a LLM server, and a language model repository. The evaluation code uses the dataset to send queries to a language model server that pulls from a language model repository, depicted in Figure 2.1.

The sequence of execution for benchmarking - once initiated by the evaluator - includes the evaluation code loading or requesting the dataset and that dataset being returned. Followed by the language model server requesting the language model from a repository, where the language model itself is returned. From this point, the evaluation code runs each query in the dataset until all queries are exhausted. The sequence of execution for benchmarking is depicted visually in Figure 2.2.

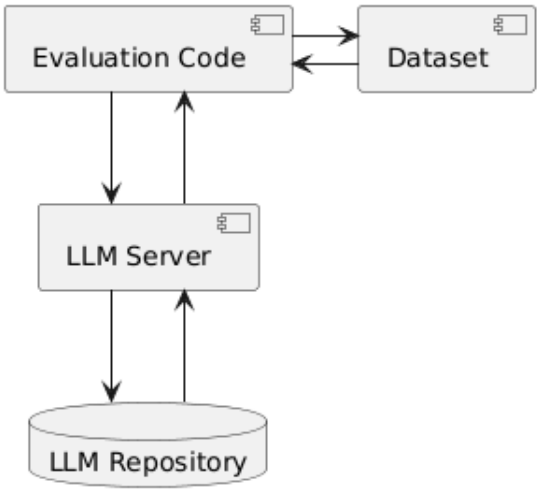

Figure 2.1. Generic Evaluation IBD

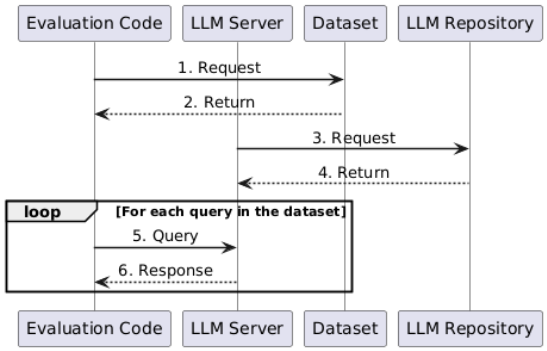

Figure 2.2. Generic Evaluation Sequence Diagram

2.2.2 Research Approach Overview

The research approach has 7 phases, including:

- Phase 1: Preparation and Baseline (MCQ)
- Phase 2: Convert MCQ to OSQ (Textual)
- Phase 3: Generate MCQ Golden Answer Variations
- Phase 4: Perform LLM Evaluations
- Phase 5: Calculate Metrics

- Phase 6: Final Integrated Evaluation Table
- Phase 7: Analyze and Interpret Results

This activity workflow shown visually in Figure 2.3. An pipeline for the process is shown in Figure 2.4. Each of the following subsections breaks down what is to occur at each of these phases.

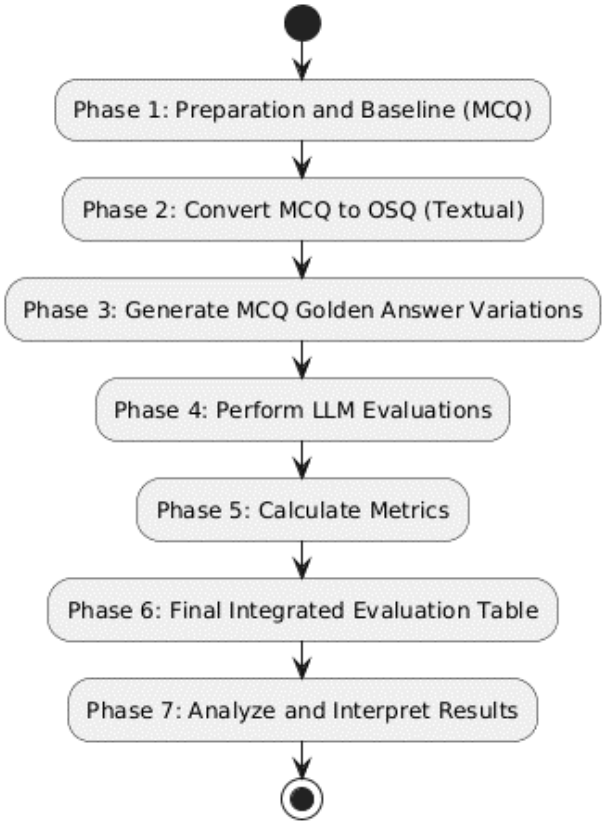

Figure 2.3. Research Approach Phases

2.2.3 Dataset Construction

Phase 1: MCQ Baseline with SysEngBench

The SysEngBench dataset is derived from foundational SE literature and is mapped to elements from the INCOSE Handbook spanning core systems engineer ing processes and

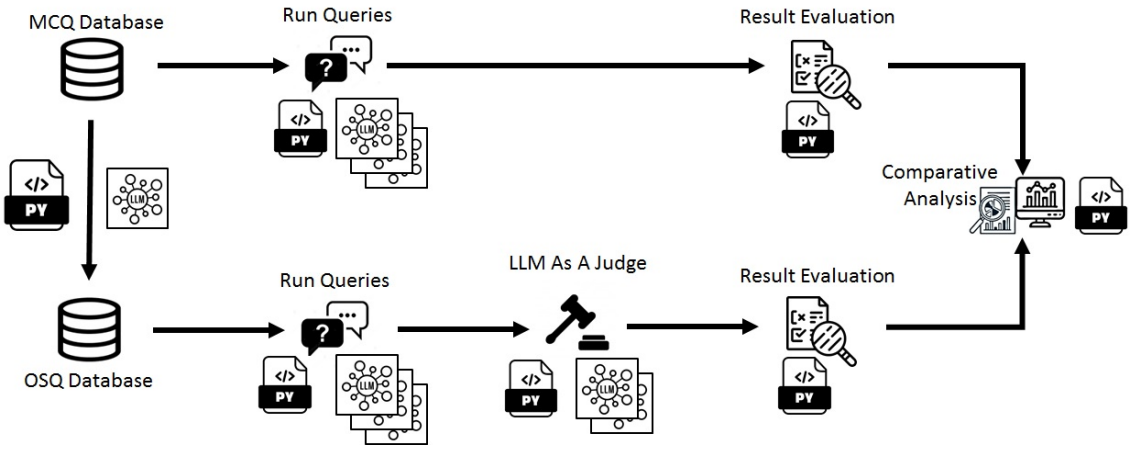

Figure 2.4. Evaluation Modality Analysis Pipeline

specialty engineering subfields [3], [10], [15] The dataset combined with a standard multiple choice format and an accuracy metric makes up the SysEngBench benchmark [3]. Each question used within the benchmark has tags for the associated INCOSE Handbook categories that are applicable to the question.

Phase 2: Multiple Choice Questions Conversion to Open Style Questions

This section focuses on the conversion from MCQ to Open Style Question (OSQ) dataset types used to benchmark different LLMs.

To address the lack of high-quality textual datasets for benchmarking, this research leverages a systematic approach for converting MCQs into OSQs. Using an LLM-assisted process, the methodology ensures semantic fidelity between the original MCQs and their OSQ equivalents. The resulting dataset is validated by human experts to verify and provide quality control when it comes to accuracy and relevance. This contribution fills a critical gap in dataset creation, enabling "apples-to-apples" comparisons between evaluation modalities.

The high level process includes using the original MCQ, evaluating for "convertibility" in the filtering stage, and then converting in the conversion stage. The high level process is shown in Figure 2.6 while the specifics of the filtering process are in Figure 2.7.

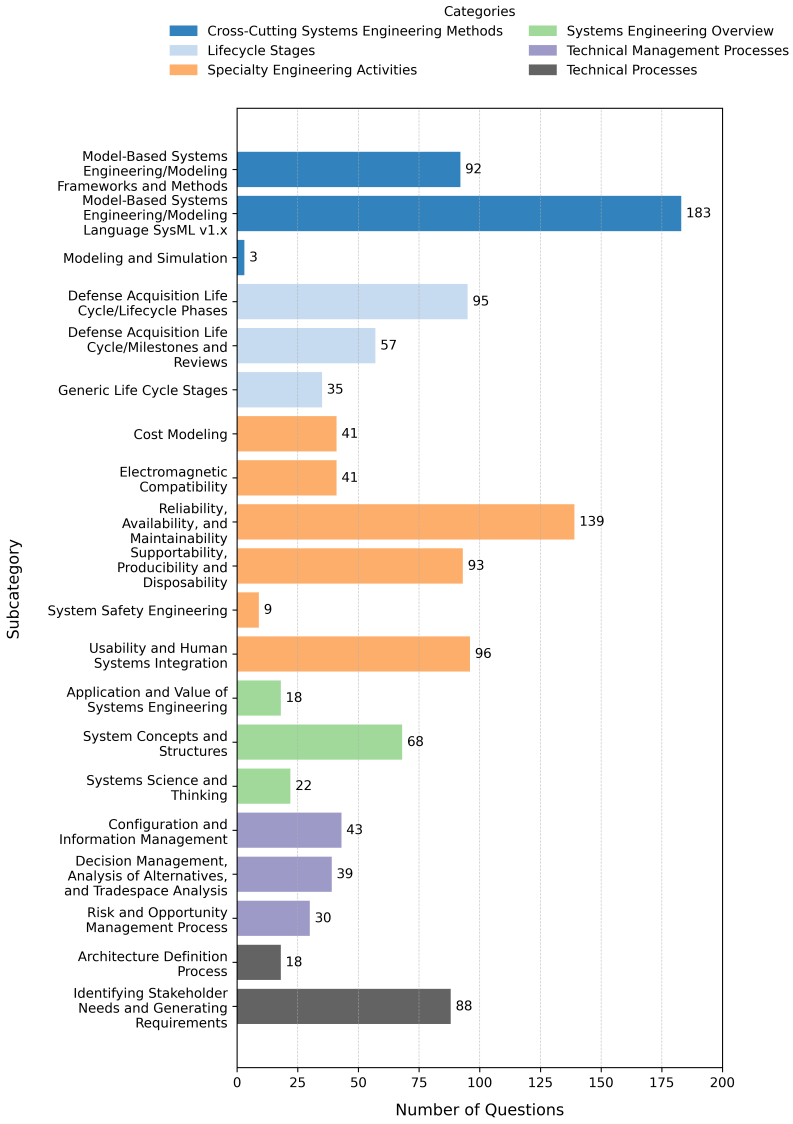

Figure 2.5. SysEngBench Benchmark Mapped to INCOSE Handbook Categories [3]

Phase 3: MCQ Distractor-Variant Datasets

Phase 3 introduces the concept of using MCQ distractor variant datasets as a part of robustness analysis. This allows the user to determine whether language models genuinely understand the domain content or merely exploit superficial patterns in the answer choices, each original MCQ from the SysEngBench dataset is systematically modified by varying

Prompt Type	Prompt
Filter Prompt	Carefully evaluate the following multiple-choice question and determine whether it can be accurately converted into an open style question format without losing its original intent, meaning, or semantic clarity. Specifically, ensure that: • The core concept being tested can be preserved in free-text form. • The question does not rely on answer choices to provide necessary context or disambiguation. • The open-ended format would still allow for a clear, concise, and unambiguous response. Answer 'Yes' if the question is suitable for conversion while maintaining intent and semantics. Otherwise, answer 'No'. In both cases, the 'Yes' or 'No' response should be followed by a confidence score between 1-10 for your confidence on whether the question can be converted or not.
Conversion Prompt	Convert the following multiple-choice question into a clear, open-ended textual question while fully preserving the original intent, meaning, and semantic clarity. Refactor the question so that it naturally prompts a textual response without requiring answer choices for context or correctness. Ensure the rephrased question tests the same knowledge and reasoning as the original multiple-choice question. Question: {question} Reference Answer: {answer} Converted Open-Ended Question: {response}

Table 2.2. Classification and Evaluation Prompts

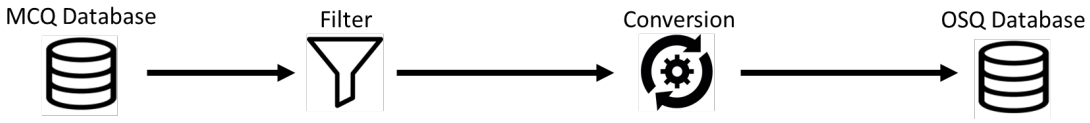

Figure 2.6. Multiple Choice Question Style to Open-Style Question Conversion Framework [4]

the position of the correct answer across different distractors (labeled options A–D). By observing fluctuations in model performance across these distractor permutations, it becomes possible to distinguish genuine knowledge from mere guessing or shallow pattern matching.

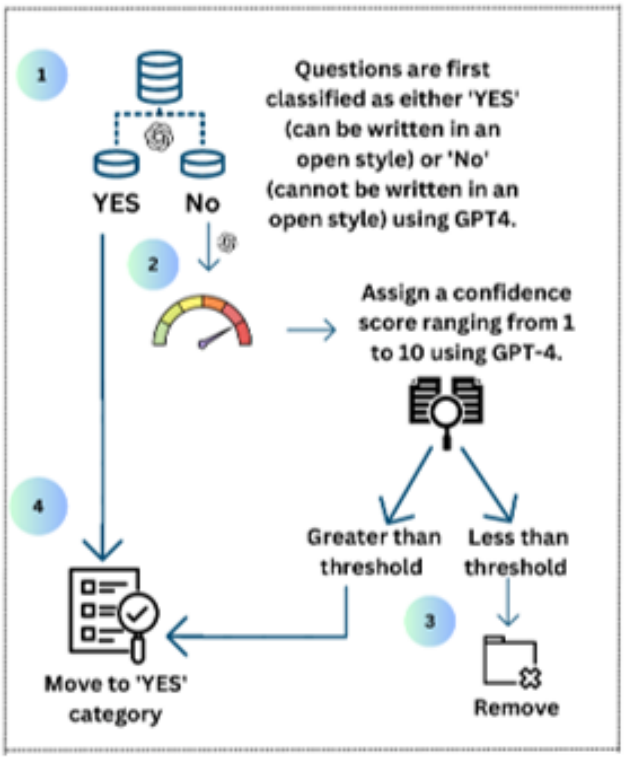

Figure 2.7. Multiple Choice Question Style to Open-Style Question Filter Process [4]

Previous research has suggested that selection bias is perhaps more common than originally thought [58].

High accuracy variance across distractor rotations indicates reliance on contextual clues or superficial strategies, whereas consistent accuracy suggests robust domain understanding. Moreover, this phase contributes towards a foundation for a principled decision-making framework for subsequent evaluations—questions where showing high variance or instability in the MCQ format may warrant deeper exploration through open-ended question (OSQ) evaluations or hybrid assessment strategies.

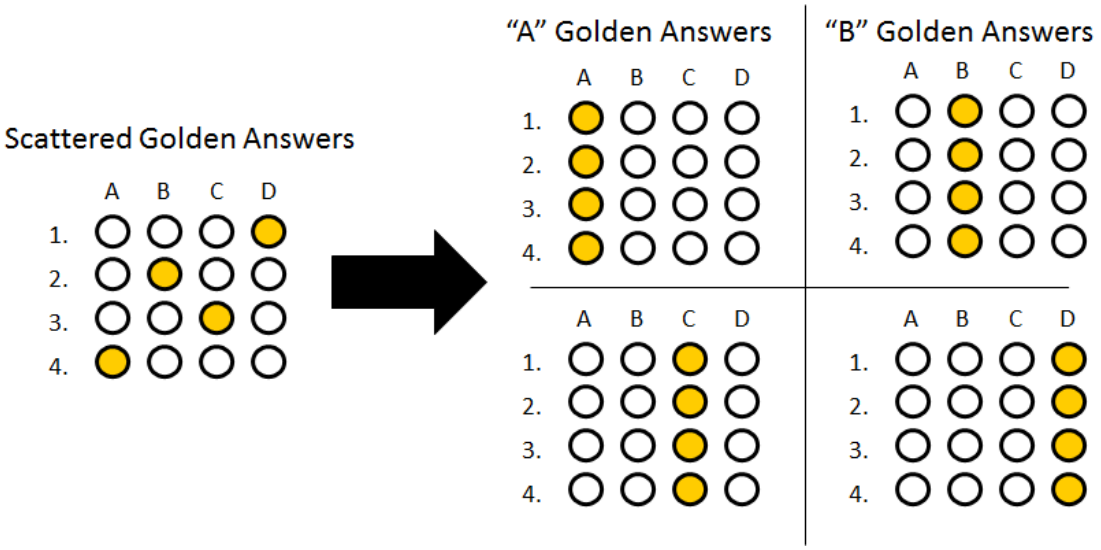

Figure 2.8. Multiple Choice Question Distractor Datasets

2.2.4 Evaluation Methods and Computations

Phase 4: LLM Evaluation Pipelines

The language model evaluation pipeline process leans on existing tools as these processes are modular and replicable. Some of the community tools available that can perform this role include: EleutherAI's lm-evaluation-harness [66], [67], LightEval [68], FastEval [69], SentEval [70], the Chatbot Arena [71], and more [72]–[74]. The most adopted community tools are the lm-evaluation-harness - also used as the backend for HuggingFace's Open Leaderboards – and the Chatbot Arena. The lm-evaluation-harness focuses on MCQs and textual outputs, evaluated programmatically while the Chatbot Arena has language models evaluated according to human feedback. For the purposes of this research, the lm-evaluation-harness was selected for its established role in the HuggingFace leaderboards, open source availability, ease of use, community support, and MCQ and OSQ friendly evaluation formats.

Phase 5: MCQ and OSQ Metric Computations

Multiple-choice questions (MCQs) serve as a fundamental method for assessing LLM performance due to their well-defined structure and objective answer choices. The evaluation of MCQ responses relies on traditional accuracy-based metrics, including accuracy, recall,

and precision. These metrics provide insight into the model's ability to select the correct answer among predefined options.

MCQ performance can be useful in measuring domain knowledge as it allows for direct assessment of whether a model can recognize and recall factual information. While this method is effective for evaluating structured knowledge, a primary hypothesis of this research is that MCQs have their limitations in capturing reasoning depth and nuanced understanding. As a result, additional evaluation techniques are necessary to assess model performance in open-ended contexts.

In contrast to MCQs, OSQs require LLMs to generate free-text responses, introducing greater complexity in evaluation. Since responses do not have predefined correct answers, LLM-based grading such as binary correct/incorrect or a scaled score from 0-100 will be used.

Cosine similarity measures the semantic closeness between a generated response and a reference answer by computing the similarity of their vector representations in a high-dimensional space. This approach allows for flexible evaluation, ensuring that responses are judged based on meaning rather than exact wording. However, cosine similarity alone does not fully capture response quality, particularly in cases where multiple valid answers exist [75].

To address this limitation, LLM-based grading is introduced as an additional evaluation method. In this approach, a separate LLM is used to assess the generated response based on a structured rubric. The rubric considers factors such as factual accuracy, conciseness, completeness, and reasoning quality. This method enables automated yet nuanced evaluation of free-text answers, ensuring that responses are graded in a way that aligns with expert human assessment.

Some LLM as a judge formats have the LLM simply grade the answer as correct or incorrect, given a golden answer [76], [77]. Some have users decide simply which response is better through pairwise evaluations [78], [79]. Others have the LLM as a judge provide a grade from 0-100 [76], [77]. In this research, a rubric for grading is proposed for LLM as a judge grading and is provided in Table 2.3.

Criterion	Score: 0-5 (Poor)	Score: 6-10 (Needs Improvement)	Score: 11-15 (Good)	Score: 16-20 (Excellent)
Accuracy and Factual Correctness	Major inaccuracies or fabrications dominate the response.	Some correct information, but several significant errors or unsupported claims are present.	Mostly accurate with minor factual issues or missing details.	Completely accurate, factually correct, and well-supported by knowledge.
Relevance and Completeness	Off-topic or fails to address the question; response is incomplete.	Some relevant content, but important aspects are missing or underdeveloped.	Mostly relevant and covers key elements with acceptable depth.	Fully relevant and comprehensive, addressing all parts of the question thoroughly.
Clarity, Coherence, and Organization	Poorly structured, incoherent, and difficult to follow.	Somewhat clear but with disorganized sections or confusing flow.	Mostly clear, logical, and easy to follow with minor structural issues.	Exceptionally clear, coherent, and well-organized throughout.
Depth of Reasoning and Insight	Lacks reasoning, analysis, or logical argumentation.	Basic reasoning present but shallow, with weak or unsupported analysis.	Solid reasoning and some analytical depth with supporting examples.	Strong, well-supported reasoning and insightful analysis throughout.
Language Quality, Grammar, and Precision	Frequent grammar errors, poor phrasing, verbose or imprecise writing.	Understandable but with noticeable grammar or language issues.	Generally well-written, concise, with minor language flaws.	Grammatically flawless, precise, professional, and stylistically strong.

Table 2.3. Notional Rubric for LLM as a Judge in Open-Style Question Evaluation

2.2.5 Phase 6 and 7 Analysis Methods

Evaluating the performance of Large Language Models (LLMs) in the context of systems engineering requires a structured and multi-faceted approach. This research employs a combination of quantitative and qualitative evaluation techniques to assess model capabilities across different question formats, including multiple-choice questions (MCQs) and open-style questions (OSQs). By leveraging a diverse set of evaluation metrics, the methodology ensures that LLM performance is analyzed in terms of accuracy, semantic alignment, and computational efficiency.

Here, we will discuss how to analyze the performance across evaluation modalities, INCOSE categories, and language models.

Evaluation Methods for Comparison Between Modalities

The comparative analysis between MCQ and OSQ modalities begins with calculating accuracy scores and variance in MCQ performance across distractor variants. High variance

Prompt Type	Prompt
Grading Prompt	You are an expert evaluator tasked with grading the model-generated response. Use the rubric (0-20 points per category, total 100): **Rubric Categories:** • **Accuracy:** Factually correct and supported? • **Relevance:** Fully addresses the question? • **Clarity:** Clear, logical, easy to follow? • **Reasoning:** Reasoning and insight? • **Language:** Well-written and precise? **Return Format:** Return your answer in the following format: Accuracy: {score}/20 – {justification} Relevance: {score}/20 – {justification} Clarity: {score}/20 – {justification} Reasoning: {score}/20 – {justification} Language: {score}/20 – {justification} Total Score: {sum}/100 **Open-Ended Question:** {question} **Model Response:** {response} **Reference Answer:** {reference}

Table 2.4. Grading Prompt with Rubric and Structured Return Format

indicates potential guessing or reliance on superficial clues rather than genuine understanding, suggesting the MCQ format alone might be insufficient for certain SE concepts. Conversely, consistently high accuracy and low variance signal reliable domain knowledge and would indicate that MCQs could be an efficient and effective assessment method for those task types.

For OSQ evaluations, each generated textual response undergoes assessment by multiple independent LLM graders using structured rubrics to maintain consistency. The inter-rater agreement among these graders is quantified using Intraclass Correlation Coefficient (ICC). High ICC values indicate strong agreement among graders and support the use of OSQs in assessing nuanced or complex reasoning tasks where single-response grading could introduce bias or instability.

The analysis then leverages hypothesis and statistical tests—including paired-sample t-tests and correlation analyses—to quantitatively compare performance across modalities. Paired-sample t-tests determine if observed differences between MCQ and OSQ scores are statistically significant within each INCOSE task category, revealing modality-specific strengths for assessing systems engineering tasks. Meanwhile, correlation analyses identify whether strong MCQ performance consistently predicts high OSQ scores, providing insights into the degree of conceptual overlap between question formats. These statistical tools collectively enable precise characterization of each modality's effectiveness, guiding decisions about modality selection for benchmarking.

Ultimately, combining these analytical approaches offers a comprehensive understanding of modality suitability across different systems engineering task categories. Categories exhibiting high MCQ variance or low ICC in OSQ evaluations might call for a need for deeper, hybrid assessment strategies. This structured comparison would then inform recommendations for modality alignment — ensuring future benchmarking and extensions to SysEngBench efforts can effectively capture and measure LLM capabilities in domain-specific systems engineering contexts.

Language Model Efficiency And Cost Analysis

Beyond accuracy performance, this research also considers the computational resources required to run LLM queries across the different evaluation modalities. The efficiency and computational cost analysis examines factors such as token count and - if available - GPU memory and utilization requirements, GPU cost, and inference speed across state-of-the-art foundational models. A simple computational cost model is built off of previous work and proposed to compare the trade-offs between model precision and computational expense in Equation 2.1 [9]. The output would offer practical recommendations for deploying LLMs in real-world systems engineering applications.

$$\text{GPU-Hours} = \frac{\text{Model}_{\text{VRAM}}}{\text{GPU}_{\text{VRAM}}} \sum_{i=1}^{N} \text{t}_i \tag{2.1}$$

Where:

- $\text{Model}_{\text{VRAM}}$: VRAM required to load the model (GB)
- GPU_{VRAM}: Memory capacity of a single GPU (GB)
- N: Total number of inferences
- t_i: i-th inference time (hours)

CHAPTER 3:
Research Contributions

This research contributes to the advancement of the evaluation of Large Language Models (LLMs) within the domain of systems engineering. It addresses key gaps in evaluation approaches, dataset creation, and the understanding of performance trade-offs for LLMs in through the comparative analysis of various benchmarking methodologies for the field. The primary focus is on comparing the evaluation modalities of MCQs and OSQs. Both accuracy and cost will be used in the comparison of the evaluation modalities.

The following contributions align with the research questions identified in Section 1.2, providing insights into the development of more effective benchmarks and the practical applications of LLMs for systems engineers. These contributions lay the groundwork for better understanding of LLM performance in systems engineering.

3.1 Task-Modality Alignment Framework for Systems Engineering Evaluation

This research introduces a structured framework that maps systems engineering task types to the evaluation modalities most appropriate for assessing them. For instance, the structured nature of MCQs provides ease of construction and objectivity, while OSQs may capture deeper reasoning and conceptual understanding. The hypothesis is that there is need for hybrid benchmarks that integrate both modalities for the field of systems engineering to ensure comprehensive evaluation. The framework provides principled guidance for aligning evaluation strategy with systems engineering task-specific demands. Additionally, this research leverages an extensible benchmark design that adapts to evolving LLM capabilities.

3.2 Robustness Analysis of MCQ Evaluation Using Distractor Variation for Systems Engineering

Expanding upon prior MCQ-only benchmarking work, this work introduces a robustness testing methodology that systematically rotates the correct answer among multiple dis-

tractors [58]. By measuring performance consistency across these variants, the analysis reveals whether models are demonstrating conceptual understanding or guessing. Accuracy variance becomes a diagnostic metric for MCQ reliability across task types.

3.3 Extension of SysEngBench with Multi-Format and Parallel Evaluation Items

This work expands the SysEngBench dataset to include both Multiple-Choice Questions (MCQs) and Open-Style Questions (OSQs) for core INCOSE defined concepts. An LLM-in-the-loop pipeline converts MCQs into free-text prompts, creating parallel evaluation paths. This dual-format design enables controlled, apples-to-apples comparisons of how different modalities capture model understanding across the same underlying SE knowledge.

3.4 Comparative Effectiveness of MCQ, OSQ, and Hybrid Evaluation Modalities

This research assesses the strengths and limitations of MCQ and OSQ formats for different SE task types. It also proposes hybrid strategies—such as combining a distractor-variant MCQ with consensus-scored OSQs to better understand evaluation accuracy. Results identify modality preferences by task type, informing more effective domain specific benchmark design.

3.5 Evidence-Based Guidance for Practical LLM Integration in Systems Engineering Workflows

This research offers actionable recommendations for applying language models in real-world SE contexts. By identifying which evaluation formats best capture model competence for different types of SE tasks, the study supports reliable AI integration into systems engineering workflows for practicing systems engineers.

CHAPTER 4:
Execution Plan

4.1 Dissertation Outline

This section provides a notional outline of the written dissertation and serves as a road-map of work needed to be accomplished. It is the student's intent to pursue an accumulation of journal articles that will cover the outline below and provide peer reviewed validation of the proposed methodology and impact to the system engineering body of knowledge.

Section ID	Chapter Title	Description
I - VII	Front Matter	• Abstract • Executive summary
1	Introduction	• Background & Motivation • Identify the Problem • Research Questions and Objectives • Research Methodology • Research Contributions, and Impact • Scope, Boundaries, and Limitations • Structure of Dissertation
2	Literature Review	• Introduction and Context Setting • Current Benchmarking Landscape • Evaluation Methods for LLMs • Unqiue Evaluation Challenges in SE and Cross-Domain Strategies • Experimental Design for Benchmark Comparison • Measurement and Analysis Techniques.
3	Materials and Methods	• Research Framework • Dataset Construction • Evaluation Methods and Computations • Analysis Metrics and Methods
4	Results	• Performance on Multiple Choice Questions • Performance on Open Style Questions • Comparison Across Modalities • Mapping to INCOSE Handbook Categories
5	Discussion	• Interpretation of Findings • Implications for Benchmarking Methodologies • Practical Applications for Systems Engineering and Insights for LLM Employment • Limitations and Future Work
6	Conclusion	• Summary of Contributions • Final Thoughts

Table 4.1. Proposed Dissertation Outline

4.2 Dissertation Schedule

This section outlines a detailed schedule of major milestones to meet the requirements for graduation in March 2026. Table 4.2 reflects the identified milestones to include a dissertation proposal, chapters, publications, defense, and graduation. Potential factors affecting the completion of the identified milestones are the iterations of each written product and schedules of other stakeholders.

It should be noted that some Task IDs have been completed slightly out of order. Due to the rapidly evolving pace of the field, submissions were completed and submitted as able to ensure first mover advantage in sharing findings.

Task ID	**Description**	**Completion Date (Month, Year)**
T.1	Develop Dissertation Outline	March 2025
T.2	Complete Dissertation Proposal	May 2025
T.3	Draft Research Questions and Objectives (Chapter 1)	May 2025
T.4	Submit Journal 1	March 2025
T.5	Complete Literature Review (Chapter 2)	July 2025
T.6	Complete Materials and Methods (Chapter 3)	August 2025
T.7	Submit Journal 2	March 2025
T.8	Conduct Experiments to get Results	September 2025
T.9	Complete Results and Discussion (Chapter 4 and 5)	October 2025
T.10	Complete Conclusion (Chapter 6)	October 2025
T.11	Complete Dissertation Formatting and Submit Initial Draft	October 2025
T.12	Defend Dissertation	November 2025
T.13	Complete Final Edits and Submit to TPO and Department Chair	December 2025
T.14	Route Final Draft	December 2025
T.15	Graduate	March 2026

Table 4.2. Proposed Dissertation Schedule

4.3 Resources and Availability Assessment

The resources required to execute the dissertation work are currently available. An assessment of resource availability is conducted monthly to address the impact on the executed

work. Any emergent resource needs are then identified, an impact analysis to the dissertation, and course of action are reported to the dissertation advisor and committee for concurrence.

The sponsoring command provides funding for tuition and travel to campus, which supports the research activities.

4.4 Notional Publications and Conference Paper

It is the intent to publish the benchmark, SysEngBench, in both MCQ and OSQ, the methodology for conversion if notable, the comparative analysis on evaluation modalities, and the results of the dissertation within peered reviewed journals and conference proceedings. It is also the PhD candidate's intention to publish as much as possible on the products of the dissertation.

Table 4.3 depicts a list of notional journal and conference publications. Work not completed is denoted by '(In Work)' preceding the working title. Work that has been submitted but under review is denoted by '(In Review)' preceding the working title. Work that has been reviewed and accepted but not yet published is denoted by '(Accepted)'. Work that has been published has no denotation.

Table 4.3. Notional Journal and Conference Publication List

Working Title	**Publication Type and Venue**	**Date**	**Author Type**
Baselining AI's Systems Engineering Performance in Cost Modeling	Presentation: Boehm CSSE Annual Research Review 2024	April 2024	Primary
Introducing SysEngBench: A Novel Benchmark for Assessing Large Language Models in Systems Engineering	Paper and Panel: NPS ARS 2024	May 2024	Primary
Leveraging Generative AI to Modify and Query MBSE Models	Paper & Panel: NPS ARS 2024	May 2024	Secondary

Working Title	Publication Type and Venue	Date	Author Type
Using AI tools for SE, from Requirements Generation and Management to Risk Identification, Analysis, and Management	Presentation: INCOSE IS 2024	July 2024	Primary
Tailored Learning for Cost Modeling: An Open Source RAG-Based Tool	Presentation: 2024 BCSSE COCOMO Forum	September 2024	Primary
(In Review) Baselining Large Language Model Performance in Systems Engineering Using SysEngBench	Paper: Wiley INCOSE Journal AI in SE Special Issue	February 2025	Primary
(In Review) Balancing Accuracy and Cost: Trade-Offs in Large Language Model Quantization for the Systems Engineering Domain	Paper: Wiley INCOSE Journal AI in SE Special Issue	February 2025	Primary
PrivateAIDELPHI: Adopting and adapting private AI for risk assessment of safety critical systems	Paper and Poster: RAMS 2025	February 2025	Secondary
A Generative AI-driven Systems Engineering Maturity and Cost Modeling Framework	Paper: CSER	April 2025	Secondary
Exploring Visual Question Answer Capabilities of Multi-Modal Large Language Models with Model Based Systems Engineering Models	Paper: NPS ARS 2025	May 2025	Secondary
Automating AI Expert Consensus: Feasibility of Language Model-Assisted Consensus Methods for Systems Engineering	Paper: NPS ARS 2025	May 2025	Primary
(In Work) Cognitive Probing of Multi-Modal LLMs: A Comparative Study of Question Formats on SysML v1 Structural Diagrams	Paper: Wiley INCOSE Journal	June 2025	Secondary
(Accepted) The Cost of Expertise: Performance Trade-Offs in Fine-Tuned LLMs for Systems Engineering	Paper: INCOSE IS 2025	July 2025	Secondary
(In Work) Evaluating Structured Model Output in LLMs: A Case Study in SysML v2	Paper: Wiley INCOSE Journal	September 2025	Secondary

THIS PAGE INTENTIONALLY LEFT BLANK

Acknowledgments

This work has benefited from the use of generative AI tools, including ChatGPT, Claude, and SciSpace, for brainstorming, writing assistance, and code development [80]–[82]. These tools were employed to enhance conceptual clarity, improve code efficiency, and support literature synthesis. All AI-generated outputs were critically reviewed, refined, and validated to ensure accuracy and alignment with academic integrity. Their contributions were limited to supporting the research process, and final responsibility for the content, analysis, and conclusions remains with the author.

THIS PAGE INTENTIONALLY LEFT BLANK

List of References

[1] "SEH 2.5 Cost Effectiveness Considerations - NASA," Mar. 2025. Available: https://www.nasa.gov/reference/2-5-cost-effectiveness-considerations/

[2] Office of the Deputy Director for Engineering, "Systems Engineering Guidebook," Dec. 2021. Available: https://ac.cto.mil/wp-content/uploads/2022/08/Systems-Eng-Guidebook_Feb2022-Cleared.pdf

[3] R. Bell, R. Longshore, R. Madachy, and A. Hanrahan, "Baselining large language model performance in systems engineering using SysEngBench," *Wiley INCOSE*, vol. Submitted, 2025.

[4] A. Myrzakhan, S. M. Bsharat, and Z. Shen, "Open-LLM-Leaderboard: From Multi-choice to Open-style Questions for LLMs Evaluation, Benchmark, and Arena," June 2024. arXiv:2406.07545 [cs]. Available: https://doi.org/10.48550/arXiv.2406.07545

[5] Y. Y. Tang and Y. Yang, "Do we need domain-specific embedding models? An empirical investigation," 2024. Available: https://doi.org/10.48550/arxiv.2409.18511

[6] T. Li *et al.*, "From crowdsourced data to high-quality benchmarks: Arena-hard and BenchBuilder pipeline," 2024. Available: https://doi.org/10.48550/arxiv.2406.11939

[7] R. Bell, R. Madachy, and R. Longshore, "_USIntroducing SysEngBench: A Novel Benchmark for Assessing Large Language Models in Systems Engineering." Acquisition Research Program, May 2024. Accepted: 2024-06-03T13:53:26Z. Available: https://dair.nps.edu/handle/123456789/5135

[8] R. Bell. "rabell/SysEngBench · Datasets at Hugging Face." Aug. 2024. Available: https://huggingface.co/datasets/rabell/SysEngBench

[9] R. Bell, R. Madachy, and R. Longshore, "Balancing accuracy and efficiency: Trade-offs in large language model quantization for the systems engineering domain," *Wiley INCOSE*, vol. Submitted, 2025.

[10] INCOSE, I. S. Council, and S. I. of Technology, "SEBoK - Guide to the Systems Engineering Body Of Knowledge." Available: https://sebokwiki.org/wiki/Guide_to_the_Systems_Engineering_Body_of_Knowledge_(SEBoK)

[11] B. Blanchard and W. Fabrycky, *Systems Engineering and Analysis*, 5th ed. Boston: Pearson, Jan. 2010.

[12] J. Boardman and B. Sauser, *Systems Thinking: Coping with 21st Century Problems*, 1st ed. Boca Raton: CRC Press, Jan. 2008.

[13] A.-K. K. P. Kludze and H. Eisner, "Engineering of complex systems: the impact of systems engineering at NASA," 2004.

[14] S. K. Son and S.-K. Kim, "The impact of systems engineering on program performance: a case study of the boeing company," in *Systems engineering advances*, 2012, pp. 537–545. Available: https://doi.org/10.1007/978-94-007-5699-1_54

[15] D. D. Walden, G. J. Roedler, K. J. Forsberg, R. D. Hamelin, and T. M. Shortell, *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 4th ed. Hoboken, N.J: John Wiley & Sons Inc, July 2015.

[16] NASA, *NASA SYSTEMS ENGINEERING HANDBOOK*, 2007.

[17] J. Tan, K. Otto, and K. Wood, "A comparison of design decisions made early and late in development," in *Proceedings of the 21st international conference on engineering design (ICED17), vol. 2: Design processes | design organisation and management*. Vancouver, Canada: Design Society / The University of British Columbia, Aug. 2017, pp. 41–50.

[18] B. Kobren. "Driving Readiness through Early Life Cycle Sustainment Planning." June 2019. Available: https://www.dau.edu/blogs/driving-readiness-through-early-life-cycle-sustainment-planning

[19] "DODAF - DOD Architecture Framework Version 2.02 - DOD Deputy Chief Information Officer," Aug. 2010. Available: https://dodcio.defense.gov/Library/DoD-Architecture-Framework/

[20] Object Management Group, "Unified Architecture Framework," 2025. Available: https://www.omg.org/uaf/

[21] J. A. Zachman, "The Zachman Framework For Enterprise Architecture," *OMG BRWG RFI*, 2003.

[22] Aiste Aleksandraviciene and Aurelijus Morkevicius, *MagicGrid Book of Knowledge*, 2nd ed. Dassault Systemes, 2021.

[23] G. O. Langford, *Engineering systems integration: Theory, metrics, and methods*. Boca Raton, FL: CRC Press, 2012. Available: https://books.google.com/books/about/Engineering_Systems_Integration.html?id=J37RBQAAQBAJ

[24] U. D. of Defense, *U.S. department of defense extension to: a guide to the project management body of knowledge (PMBOK guide)*. Fort Belvoir, Virginia, USA: Defense Acquisition University Press, June 2003. tex.version: 1.0. Available: https://www.dau.edu/library

[25] I. of Electrical and E. Engineers, *IEEE standard for application and management of the systems engineering process*. New York, NY, USA: IEEE, 1999. Available: https://ieeexplore.ieee.org/document/796669

[26] DAU, "Model-Based Systems Engineering," 2024. Available: https://www.dau.edu/datl/b/model-based-systems-engineering

[27] A. L. Ramos, J. V. Ferreira, and J. Barceló, "Model-Based Systems Engineering: An Emerging Approach for Modern Systems," *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 42, no. 1, pp. 101–111, Jan. 2012. Conference Name: IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews). Available: https://doi.org/10.1109/TSMCC.2011.2106495

[28] J. M. Borky and T. H. Bradley, *Effective Model-Based Systems Engineering*, 1st ed. Cham: Springer, Sep. 2018.

[29] Vaneman, "Webinar: Model-Based Systems Engineering De-mystified," Mar. 2020. Available: https://www.youtube.com/watch?v=BPlphC88xR4

[30] L. Delligatti, *SysML Distilled: A Brief Guide to the Systems Modeling Language*, 1st ed. Upper Saddle River, NJ Munich: Addison-Wesley Professional, Nov. 2013.

[31] A. Vaswani *et al.*, "Attention is All you Need," in *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2017, vol. 30. Available: https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[32] Lab42, "About ARC," 2024. Available: https://lab42.global/arc/

[33] Microsoft, "Phi Open Models - Small Language Models." Available: https://azure.microsoft.com/en-us/products/phi

[34] "Llama 3.2: Revolutionizing edge AI and vision with open, customizable models." Available: https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/

[35] A. Myrzakhan, S. M. Bsharat, and Z. Shen, "Open-LLM-Leaderboard: From Multi-choice to Open-style Questions for LLMs Evaluation, Benchmark, and Arena," June 2024. arXiv:2406.07545 [cs] version: 1. Available: https://doi.org/10.48550/arXiv.2406.07545

[36] J. A. Estefan, "Survey of Model-Based Systems Engineering (MBSE) Methodologies," 2008.

[37] D. Harvey, P. Logan, M. Waite, and T. Liddy, "Document the model, don't model the document," Jan. 2012.

[38] "Department of Defense Digital Engineering Strategy," 2018.

[39] "DOD Digital Modernization Strategy 2019," 2019.

[40] M. T. W. Simms, "STATUS OF ADOPTION AND IMPLEMENTATION OF DIGITAL ENGINEERING INFRASTRUCTURE AND WORKFORCE DEVELOPMENT WITHIN THE DEPARTMENT OF DEFENSE," Oct. 2022.

[41] J. L. Jones, "United States Navy and Marine Corps Digital Systems Engineering Transformation Strategy," 2020.

[42] B. R. Brown, *Engineering intelligent systems: Integrating AI into systems engineering*. Wiley, 2023. Available: https://www.amazon.com/Engineering-Smarter-Systems-Approaches-Intelligence/dp/1119665590

[43] E. Sanu, T. K. Amudaa, P. Bhat, G. Dinesh, A. U. K. Chate, and R. K. P, "Limitations of large language models," pp. 1–6, 2024. Available: https://doi.org/10.1109/csitss64042.2024.10817070

[44] A. Matarazzo and R. Torlone, "A survey on large language models with some insights on their capabilities and limitations," 2025. Available: https://doi.org/10.48550/arxiv.2501.04040

[45] S. Johnson and D. Hyland-Wood, "A primer on large language models and their limitations," 2025. Available: https://doi.org/10.32388/nhjyvs

[46] OpenAI, "Introducing OpenAI o1," Sep. 2024. Available: https://openai.com/o1/

[47] OpenAI, "OpenAI o3-mini." Available: https://openai.com/index/openai-o3-mini/

[48] DeepSeek-AI *et al.*, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," Jan. 2025. arXiv:2501.12948 [cs]. Available: https://doi.org/10.48550/arXiv.2501.12948

[49] J. Li *et al.*, "Benchmarking large language models in evidence-based medicine," *IEEE Journal of Biomedical and Health Informatics*, pp. 1–14, 2024. Available: https://doi.org/10.1109/jbhi.2024.3483816

[50] M. S. Ünal, "Holistic evaluation of language models," *Annals of the New York Academy of Sciences*, 2023. Available: https://doi.org/10.1111/nyas.15007

[51] Y. Liu, C. Tantithamthavorn, Y. Liu, and L. Li, "On the reliability and explainability of language models for program generation," *ACM Transactions on Software Engineering and Methodology*, 2023. Available: https://doi.org/10.1145/3641540

[52] Q. Jin, B. Dhingra, Z. Liu, W. Cohen, and X. Lu, "PubMedQA: A Dataset for Biomedical Research Question Answering," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 2567–2577. Available: https://doi.org/10.18653/v1/D19-1259

[53] A. Pal, L. K. Umapathi, and M. Sankarasubbu, "MedMCQA : A Large-scale Multi-Subject Multi-Choice Dataset for Medical domain Question Answering," Mar. 2022. arXiv:2203.14371 [cs]. Available: https://doi.org/10.48550/arXiv.2203.14371

[54] N. Guha *et al.*, "LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models," Aug. 2023. arXiv:2308.11462 [cs]. Available: https://doi.org/10.48550/arXiv.2308.11462

[55] I. Chalkidis *et al.*, "LexGLUE: a benchmark dataset for legal language understanding in English," in *Proceedings of the 60th annual meeting of the association for computational linguistics (volume 1: Long papers)*. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 4310–4330. Available: https://aclanthology.org/2022.acl-long.297

[56] S. Xue, T. Chen, F. Zhou, Q. Dai, Z. Chu, and H. Mei, "FAMMA: A Benchmark for Financial Domain Multilingual Multimodal Question Answering," Oct. 2024. arXiv:2410.04526 [cs]. Available: https://doi.org/10.48550/arXiv.2410.04526

[57] P. Islam, A. Kannappan, D. Kiela, R. Qian, N. Scherrer, and B. Vidgen, "FinanceBench: A New Benchmark for Financial Question Answering," Nov. 2023. arXiv:2311.11944 [cs]. Available: https://doi.org/10.48550/arXiv.2311.11944

[58] C. Zheng, H. Zhou, F. Meng, J. Zhou, and M. Huang, "LARGE LANGUAGE MODELS ARE NOT ROBUST MULTIPLE CHOICE SELECTORS," 2024.

[59] T. Fu, J. Conde, G. Martínez, M. Grandury, and P. Reviriego, "Multiple Choice Questions: Reasoning Makes Large Language Models (LLMs) More Self-Confident Even When They Are Wrong," Jan. 2025. arXiv:2501.09775 [cs]. Available: https://doi.org/10.48550/arXiv.2501.09775

[60] G. Góral and E. Wiśnios, "When All Options Are Wrong: Evaluating Large Language Model Robustness with Incorrect Multiple-Choice Options," Aug. 2024. arXiv:2409.00113 [cs] version: 1. Available: https://doi.org/10.48550/arXiv.2409.00113

[61] S. Banerjee, A. Agarwal, and E. Singh, "The vulnerability of language model benchmarks: Do they accurately reflect true LLM performance?" 2024. tex.publication_type: article. Available: https://doi.org/10.48550/arxiv.2412.03597

[62] R. Madachy, R. Bell, and R. Longshore, "Systems Acquisition Cost Modeling Initiative for AI Assistance." Naval Postgraduate School, May 2024.

[63] R. Madachy, R. Longshore, and R. Bell, "Systems and software engineering cost modeling of AI assistance," Ireland, 2024. tex.topics: Academia (curricula, course life cycle, etc.), Aerospace, Project Planning, Project Assessment, and/or Project Control, Life-Cycle Costing and/or Economic Evaluation, Artificial Intelligence, Machine Learning, Defense.

[64] R. Madachy, R. Bell, and R. Longshore, "A generative AI-driven systems engineering maturity and cost modeling framework," in *Proceedings of the 22nd annual conference on systems engineering research (CSER)*, A. M. Madni, D. Verma, M. Sievers, M. Wheaton, E. Ordoukhanian, and S. Purohit, Eds. Long Beach, CA, USA: University of Southern California and Stevens Institute of Technology, Mar. 2025, vol. In Press.

[65] R. Madachy, R. Bell, and R. Longshore, "A generative AI-driven systems engineering maturity and cost modeling framework," in *Proceedings of the 2025 conference on systems engineering research* (Procedia Computer Science), 2025. tex.publication_type: inproceedings.

[66] EleutherAI, "EleutherAI/lm-evaluation-harness," Aug. 2024. original-date: 2020-08-28T00:09:15Z. Available: https://github.com/EleutherAI/lm-evaluation-harness

[67] L. Gao *et al.*, "A framework for few-shot language model evaluation," July 2024. tex.version: v0.4.3. Available: https://doi.org/10.5281/zenodo.12608602

[68] "huggingface/lighteval," Aug. 2024. original-date: 2024-01-26T13:15:39Z. Available: https://github.com/huggingface/lighteval

[69] "FastEval/FastEval," Aug. 2024. original-date: 2023-05-07T17:08:45Z. Available: https://github.com/FastEval/FastEval

[70] "facebookresearch/SentEval," Aug. 2024. original-date: 2017-05-18T20:44:47Z. Available: https://github.com/facebookresearch/SentEval

[71] W.-L. Chiang *et al.*, "Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference," Mar. 2024. arXiv:2403.04132 [cs]. Available: https://doi.org/10.48550/arXiv.2403.04132

[72] R. Bommasani, D. Zhang, T. Lee, and P. Liang, "Improving Transparency in AI Language Models: A Holistic Evaluation," Feb. 2023.

[73] X. Liu *et al.*, "AgentBench: Evaluating llms as agents," *arXiv preprint arXiv: 2308.03688*, 2023. Available: https://llmbench.ai/agent

[74] MLflow. "MLflow." Available: https://mlflow.org/

[75] K. Zhou, K. Ethayarajh, D. Card, and D. Jurafsky, "Problems with cosine as a measure of embedding similarity for high frequency words," in *Annual meeting of the association for computational linguistics*, 2022, vol. abs/2205.05092. tex.publication_type: inproceedings. Available: https://doi.org/10.48550/arXiv.2205.05092

[76] H. Li *et al.*, "LLMs-as-Judges: A Comprehensive Survey on LLM-based Evaluation Methods," Dec. 2024. arXiv:2412.05579 [cs]. Available: https://doi.org/10.48550/arXiv.2412.05579

[77] J. Gu *et al.*, "A Survey on LLM-as-a-Judge," Mar. 2025. arXiv:2411.15594 [cs]. Available: https://doi.org/10.48550/arXiv.2411.15594

[78] L. Zheng *et al.*, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," Dec. 2023. arXiv:2306.05685 [cs]. Available: https://doi.org/10.48550/arXiv.2306.05685

[79] R. Hu *et al.*, "Training an LLM-as-a-Judge Model: Pipeline, Insights, and Practical Lessons," Feb. 2025. arXiv:2502.02988 [cs]. Available: https://doi.org/10.1145/3701716.3715265

[80] OpenAI. "OpenAI." 2025. Available: https://openai.com/

[81] Anthropic AI, "Anthropic Home," 2025. Available: https://www.anthropic.com/

[82] SciSpace, "SciSpace," 2025. Available: https://typeset.io/